Flower Supply
Servlet Application

Student: *Vararean Diana-Lorena*
Group: *244*

October 25, 2026

Contents

1 Introduction **2**

2 Architecture **2**

3 Technologies and System Calls **2**

4 Data Model and Storage **3**

5 Warehouse Service **3**

6 Shop Service **4**

7 Processing Sequence **5**

8 Deployment and Examples **6**

9 Testing **7**

10 Additional Solution **7**

1 Introduction

Flower Supply is a small supply application implemented with Jakarta Servlets. It models one flower shop that requests stock from one central warehouse.

The application consists of two independent Maven web projects. The Warehouse service runs in Tomcat and owns the central stock, while the Shop service runs in Jetty and owns the local shop stock. The Shop communicates with the Warehouse through a synchronous HTTP/JSON request. A request is immediately approved or rejected and the Shop updates its local stock only after obtaining an approval. This implementation relies on Tomcat and Jetty as the servlet containers for deployment and execution.

2 Architecture

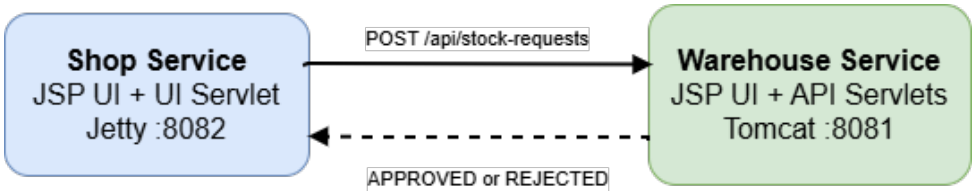


Figure 1: System Architecture Overview

Each service is packaged as a separate WAR file. The services do not share Java classes or in-memory state. The Warehouse exposes the only public JSON API. The Shop exposes UI pages only.

3 Technologies and System Calls

- Java 21 and Maven WAR packaging;
- Jakarta Servlet API 6.0;
- Tomcat for the Warehouse and Jetty for the Shop;
- JSP and JSTL for server-rendered user interfaces;
- Jackson for JSON serialization and deserialization;
- `java.net.http.HttpClient` for the synchronous service call;
- `ConcurrentHashMap` for in-memory stock storage;
- HTTP system calls through the Java HTTP client and command-line `curl` during verification.

No data is written at runtime. At startup, each application reads its own CSV file from the classpath. A restart therefore resets the stock to the CSV values.

4 Data Model and Storage

The `Flower` object has three properties:

- `flowerId: String`
- `name: String`
- `quantity: int`

The CSV format is the following:

```
flowerId,name,quantity
rose-red,Red Rose,200
```

The Warehouse and Shop each have a separate `StockStore` singleton. The Warehouse uses an atomic synchronized approval operation. It checks existence and available quantity and decrements the flower in the same critical section. The Shop uses synchronized increase and decrease operations so that a local stock update cannot be partially applied.

5 Warehouse Service

Objects and method prototypes.

Object	Public constructors and methods

`Flower`	`Flower()`
	`Flower(String flowerId, String name, int quantity)`
	`String getFlowerId()`
	`String getName()`
	`int getQuantity()`
	setters for the three properties

`StockStore`	`static StockStore getInstance()`
	`void loadFromCsv(InputStream input) throws IOException`
	`List<Flower> findAll()`
	`boolean approveRequest(String flowerId, int requestedQuantity)`

`WarehouseStartupListener`	`void contextInitialized(ServletContextEvent event)`
`StockServlet`	`void doGet(HttpServletRequest request, HttpServletResponse response) throws IOException`
`StockRequestServlet`	`void doPost(HttpServletRequest request, HttpServletResponse response) throws IOException`

Object	Public constructors and methods
`WarehouseUiServlet`	`void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException`

API contract.

Request	Processing and output
`GET /warehouse/api/stock`	Returns HTTP 200 and a JSON array containing the current warehouse stock.
`POST /warehouse/api/stock-requests`	Receives `{"flowerId": String, "quantity": int}`. For a valid request, returns HTTP 200 and either `{"status":"APPROVED"}` or `{"status":"REJECTED"}`.
Invalid request	Malformed JSON, missing fields, empty flower identifier, or `quantity <= 0` returns HTTP 400.

6 Shop Service

Objects and method prototypes.

Object	Public constructors and methods

`StockStore`	`static StockStore getInstance()`
	`void loadFromCsv(InputStream input) throws IOException`
	`List<Flower> findAll()`
	`boolean increase(String flowerId, int quantity)`
	`boolean decrease(String flowerId, int quantity)`

`WarehouseClient`	`StockRequestResult requestStock(String flowerId, int quantity)`

`StockRequestResult`	`StockRequestResult(Status status, String message)`
	`Status status()`
	`String message()`

`ShopStartupListener`	`void contextInitialized(ServletContextEvent event)`
`ShopServlet`	`void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException` `void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException`

User interface. The Shop JSP displays the current stock in a table. Flowers with quantity strictly below 10 receive a red visual highlight and a `LOW` label. The page provides two forms:

- a warehouse request form containing a flower and a positive quantity;
- a local decrease form simulating a sale.

The Shop uses a two-second connection timeout and a five-second request timeout. If the Warehouse cannot be reached, the page shows `Warehouse is unavailable` and does not change local stock. No retry is performed.

7 Processing Sequence

The following processing diagram documents the main path from a user submission to the final page.

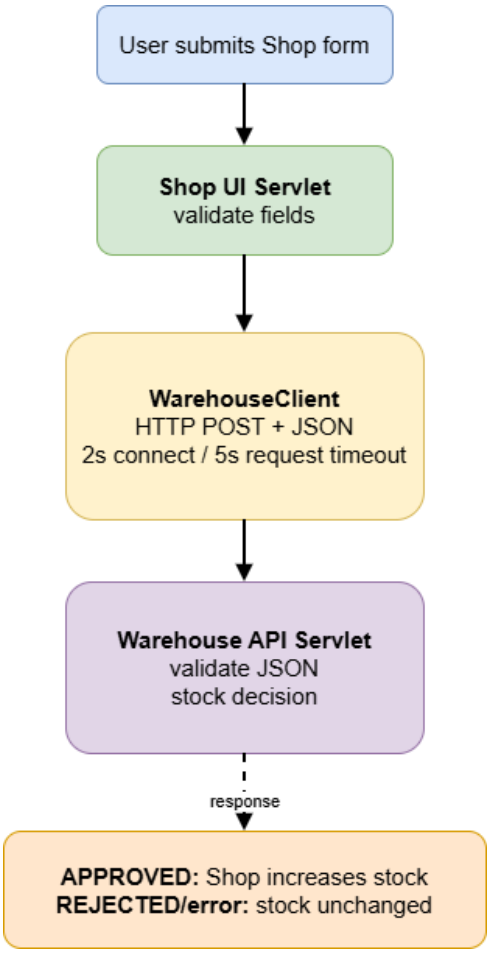


Figure 2: Synchronous request processing

The following steps outline the synchronous request processing in detail:

1. The user submits the Shop request form.
2. `ShopServlet` validates the form values.

3. `WarehouseClient` sends an HTTP POST with JSON.

4. `StockRequestServlet` validates the JSON and atomically checks the available Warehouse stock.

5. The Warehouse processes the request and returns a decision:

   - **Approved:** If stock is sufficient, the Warehouse decrements its inventory and returns `APPROVED`. The Shop then increases its local quantity.
   - **Rejected:** If the flower is unknown or stock is insufficient, the Warehouse returns `REJECTED`. The Shop's local stock remains unchanged.

6. The Shop renders the updated page to the user.

If the HTTP call times out, the Warehouse cannot be reached, or the response is invalid, the client reports an error and the local Shop stock remains unchanged.

8 Deployment and Examples

The deployment is manual, and the project produces two independent WAR files. The Warehouse WAR is copied to the Tomcat `webapps` directory and the Shop WAR is copied to the configured Jetty deployment directory. The `warehouse.war` produces the `/warehouse` context and `shop.war` produces the `/shop` context.

Build and deployment sequence.

1. Verify that Java 21, Maven, Tomcat, and Jetty are installed.

2. Run the unit tests and build each WAR:

   ```
   cd warehouse-service && mvn clean test package
   cd ../shop-service && mvn clean test package
   ```

3. Copy `warehouse-service/target/warehouse.war` to the Tomcat deployment directory.

4. Copy `shop-service/target/shop.war` to the Jetty deployment directory.

5. Start both application servers and wait for their deployment scanners to load the WAR files.

6. Verify the UI and the Warehouse JSON endpoints.

Project lifecycle script. `run.sh` is located in the project root and delegates server lifecycle operations to the existing scripts in the parent `servers` directory. It calculates paths relative to its own location, so it can be called from any current directory. The available commands are:

Command	Operation
`./run.sh build`	Run Maven tests and create both WAR files.

`./run.sh deploy`	Build, copy both WAR files, restart both servers, and wait until both HTTP URLs respond.
`./run.sh start`	Start Tomcat and Jetty, then wait for readiness.
`./run.sh stop`	Stop Jetty and Tomcat using their stop scripts.
`./run.sh restart`	Stop and start both application servers.
`./run.sh status`	Probe the two UI URLs and report running or stopped.
`./run.sh test`	Run the integration script against both deployed applications.

The Warehouse API can be tested with:

```
curl -X POST http://localhost:8081/warehouse/api/stock-requests \
  -H 'Content-Type: application/json' \
  -d '{"flowerId":"rose-red","quantity":50}'
```

The expected successful response is:

```
{"status":"APPROVED"}
```

9 Testing

Unit tests cover CSV loading, valid and invalid Warehouse requests, stock decrease and increase operations, and the strict low-stock boundary: quantity 9 is low stock while quantity 10 is not. Integration tests cover both UIs, the approved and rejected flows, malformed requests, unavailable Warehouse behavior, simultaneous server execution, and reset after restart.

10 Additional Solution

The implemented solution uses synchronous HTTP/JSON because the assignment requires an immediate decision. A message queue would decouple the services and support asynchronous processing. A larger REST client layer could provide versioned API contracts, but the current two-endpoint API is sufficient for one Shop and one Warehouse.